CIS*3110 Operating Systems

Week 5

Lecture #9 Virtualization: More Paging

Prof. Yulia Kotseruba

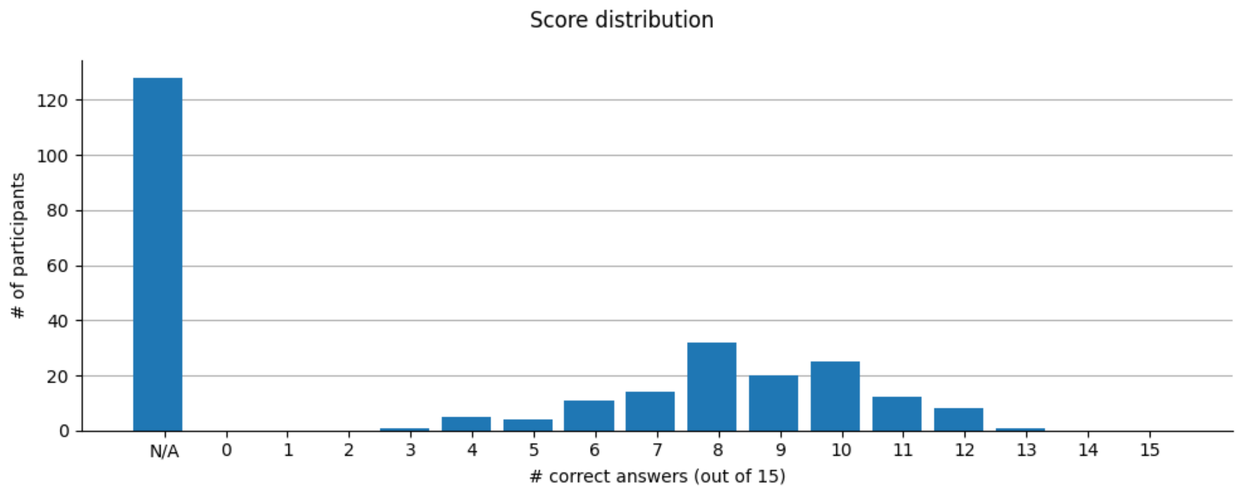
February 5, 2025

Quiz 7

- **61%** average (up from 50% in quiz 6)
- 4 hardest questions:
  - (21%) What is the output of this code? a) ppid=1 b) ppid=0 c) nothing d) segfault
  - (22%) What is the result of this code? a) compiler error b) segmentation error c) x=42 d) x=42 x=42
  - (25%) What will this code print if executed by a regular user? a) priority=0 b) priority=3 c) priority = 10 d) nothing
  - (38%) Which process has a PID of 1? a) any kernel process b) orphan process c) zombie process d) none of the above

```c
int main() {
    if (fork() > 0){
        void *p; free(p);
    }
    sleep(1);
    printf("ppid=%d\n", getppid());
}
```

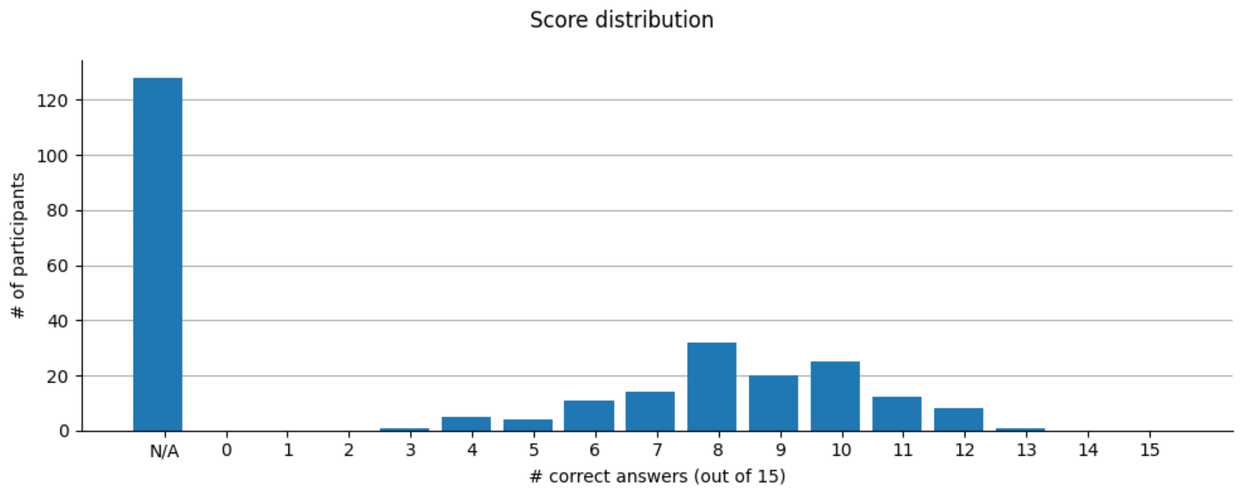
```c
int main() {
    int x = 42;
    printf("x=%d ", x);
    printf("x=%s ", x);
}
```



Quiz 7

```
int main() {
    setpriority(PRIO_PROCESS, 0, 3);
    if (fork()==0 && setpriority(PRIO_PROCESS,0,0)==0)
        printf("priority=%d\n", getpriority(PRIO_PROCESS, 0));
    return 0;
}
```

- **61%** average (up from 50%
- 4 hardest questions:
  - (21%) What is the output of this code? a) ppid=1 b) ppid=0 c) nothing d) segfault
  - (22%) What is the result of this code? a) compiler error b) segmentation error c) x=42 d) x=42 x=42
  - (25%) What will this code print if executed by a regular user? a) priority=0 b) priority=3 c) priority = 10 d) nothing
  - (38%) Which process has a PID of 1? a) any kernel process b) orphan process c) zombie process d) none of the above



Recap

- **Paging** splits memory into **equally-sized** chunks — simpler and more efficient memory allocation
  - Pros
    - fixed-size pages avoid **external fragmentation**
    - **easy to allocate** memory (all blocks are the same, any free one will do)
    - does **not** require hardware support - flexible
  - Cons
    - does not eliminate **internal fragmentation** (if page size is too big)
    - page tables are **large**
    - accessing memory is **slow** (**doubles** the number of memory operations per instruction) -> TLB helps with this

Recap

- **Translation Lookaside Buffer (TLB)** - memory **cache** for storing translations of virtual memory to physical memory addresses
  - Pros
    - **reduces** memory accesses to page table
    - **very fast** hardware-supported implementation
  - Cons
    - needs **careful management** to avoid too many **TLB misses**

Recap

page	offset
n bits	m bits

- physical memory: 2^4 - 16 addresses
- virtual memory: 2^4 - 16 addresses
- How to decide on the page table size (n)?
  - n = 0 - no page table, just address the physical memory

0	0	0	1

0 bits page 4 bits offset

addresses all virtual memory

here virtual memory is the same size as physical for simplicity

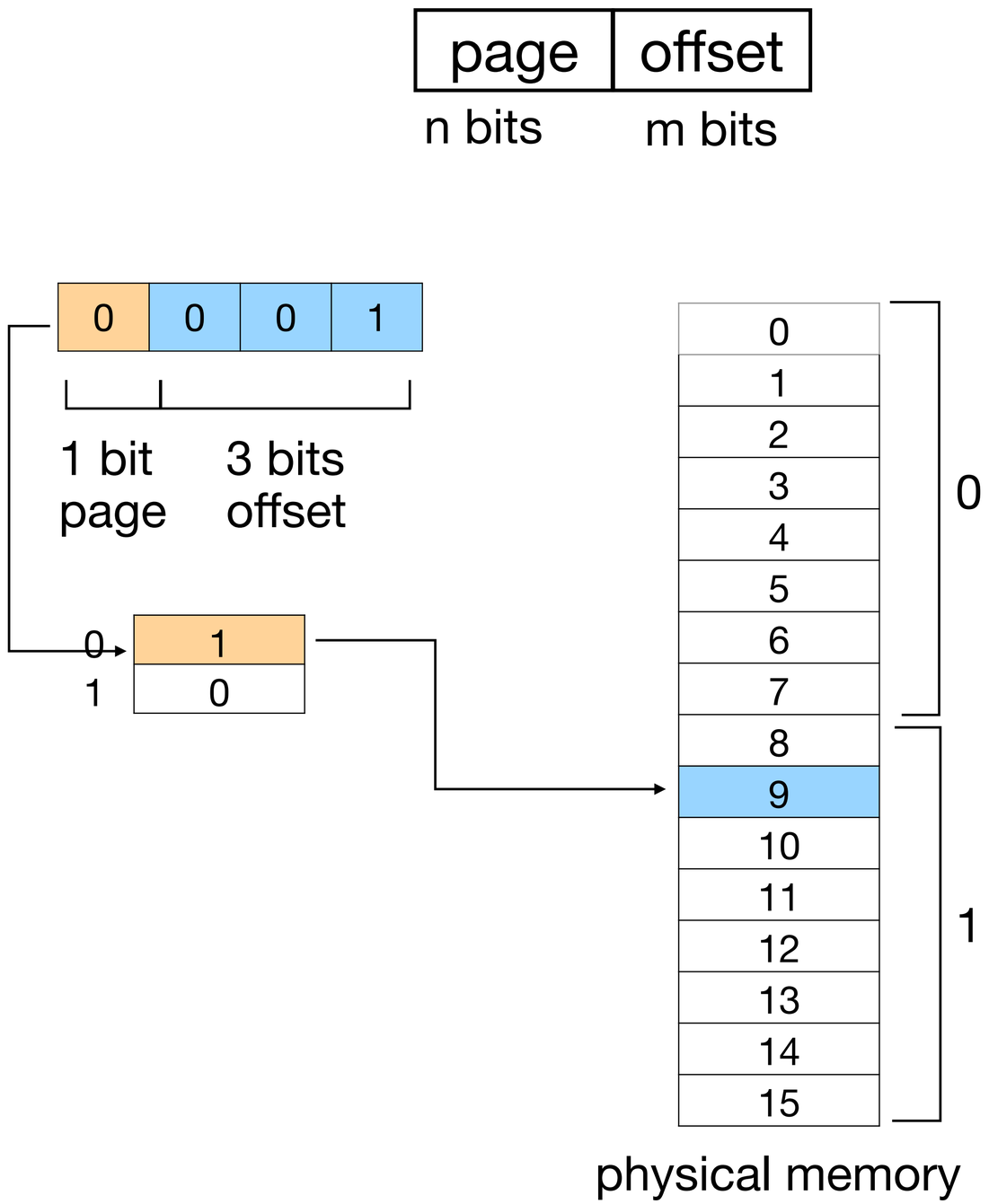
in reality, virtual memory is **a lot** larger than RAM

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

physical memory

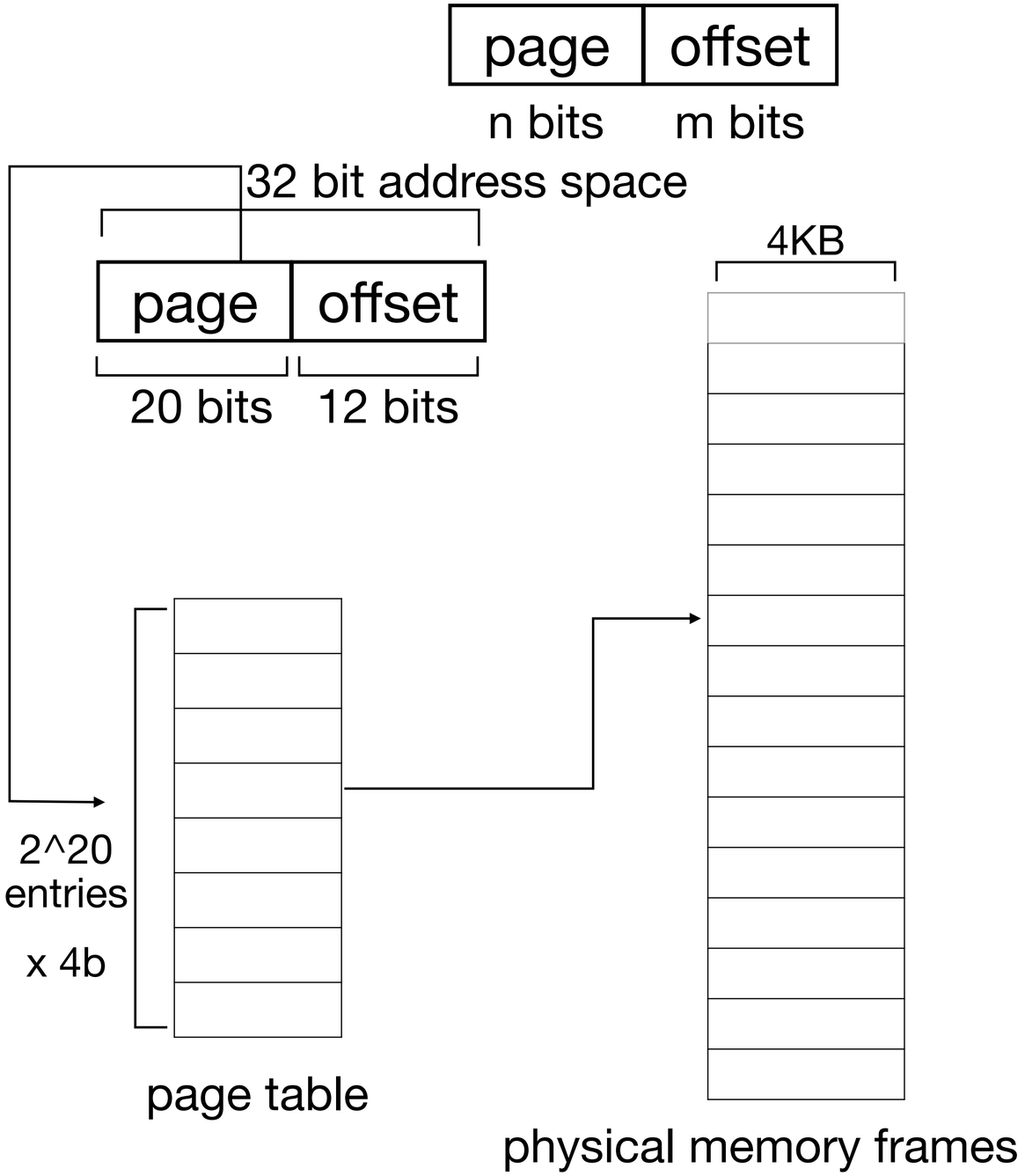
Recap

- physical memory: 2^4 - 16 addresses
- virtual memory: 2^4 - 16 addresses
- How to decide on the page table size (n)?
  - n = 1 - page table with 2 entries
    - page size is 8 - 3 bits to address
    - page table size is 2 - 1 bit to address



Recap

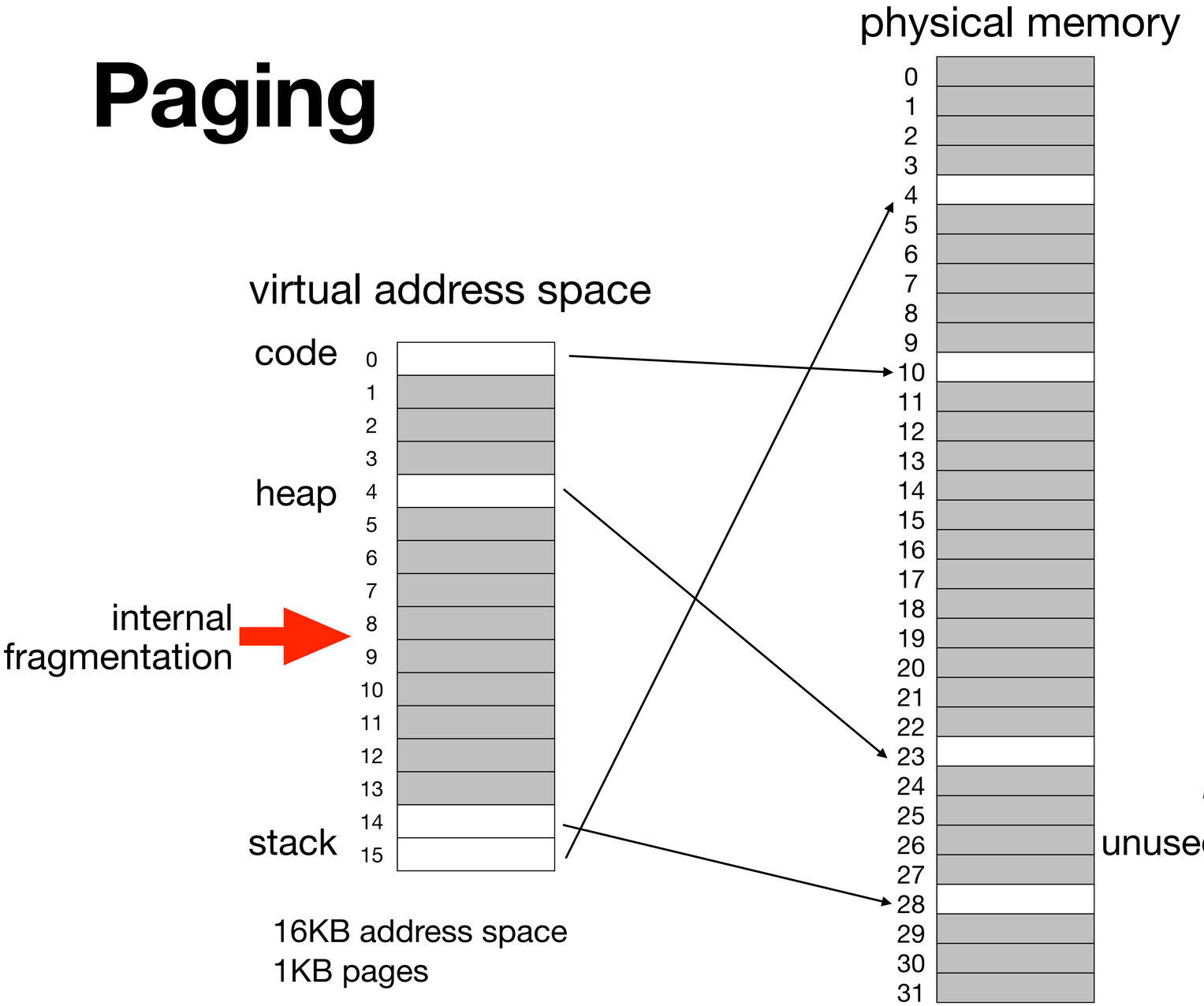
- Page tables in our examples are **tiny**
- Real page tables are **large**
  - e.g. 32-bit address space with 4KB-sized pages
    - will need 12 bits to address 4KB=4096=2^12 offsets
    - 20 bits for virtual pages = ~1M (2^20) translations in the table
    - each entry is 4 bytes -> 4MB memory



Page tables

- Problem: Page tables consume too much memory
- Option #1: **larger pages**
  - Pros: smaller page tables
  - Cons: internal fragmentation
- Most OSs use small-ish pages - you can check
  - Mac OS - `pagesize` (16KB on my laptop)
  - Linux - `getconf PAGE_SIZE` (4KB on our server)
  - Windows - in PowerShell type `[System.Environment]::SystemPageSize` (I got 4KB)
- Intel x86_64 supports 4KB, 2MB and 1GB page sizes (also read about huge_pages in Linux)

Paging



page table

PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
23	1	rw-	1	1
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
28	1	rw-	1	1
4	1	rw-	1	1

paging does not automatically eliminate internal fragmentation

Page tables

- Problem: Page tables are too big and use too much memory
- Option #1: larger pages
- Option #2: **segmentation for page tables**

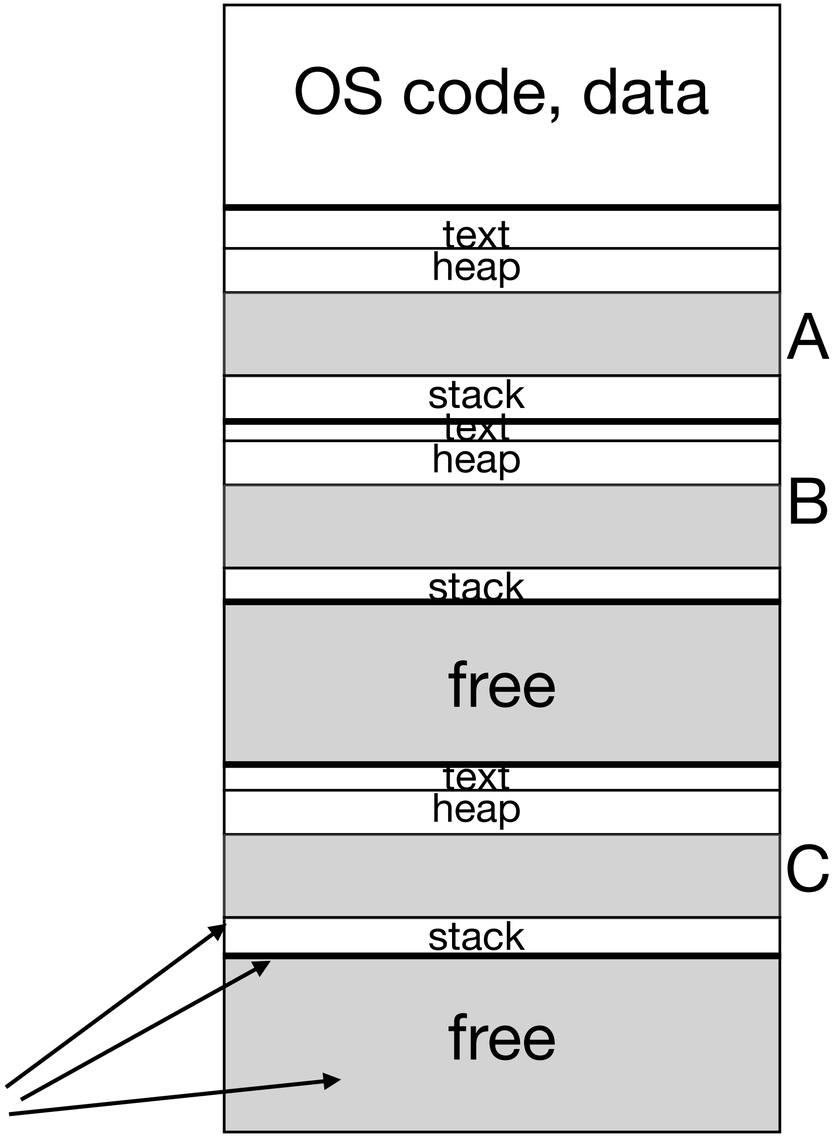
Segmentation

- 3 segments - text, heap, stack
- 3 pairs of base & bounds registers
- Addressing
  - Find segment from virtual address
  - Find offset from virtual address
  - If offset > bounds[segment] -> abort
  - Otherwise, phys_address = base[segment] + offset

virtual address

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	1	1	0	1	0	0	0

segment offset

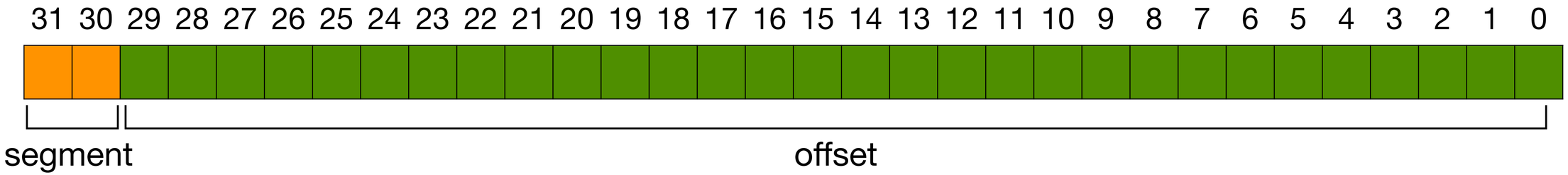


Paging + segmentation

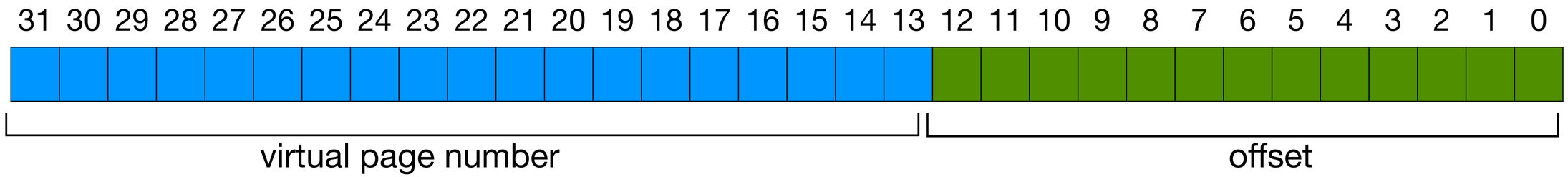
- **Hybridize** paging and segmentation
  - instead of one page table for address space
  - **3 page tables**: for code, heap and stack
  - base register - physical address of the page table for the segment
  - bounds register - number of valid entries in the table for the segment

Paging + segmentation

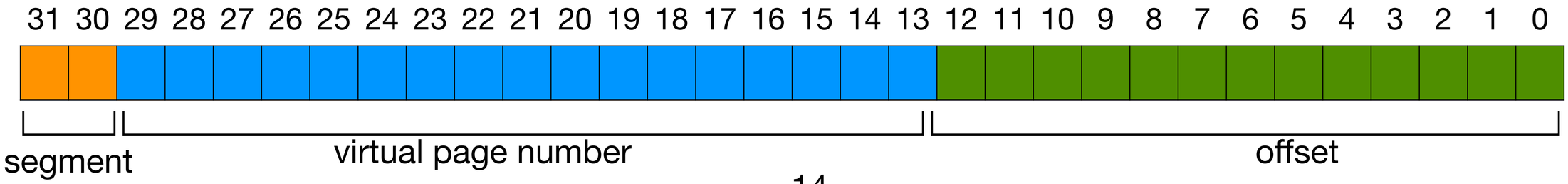
segmentation



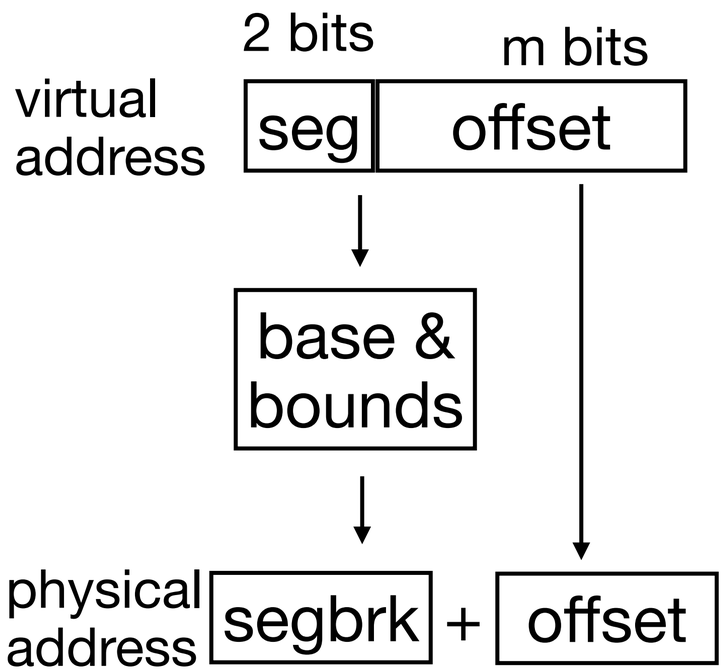
paging



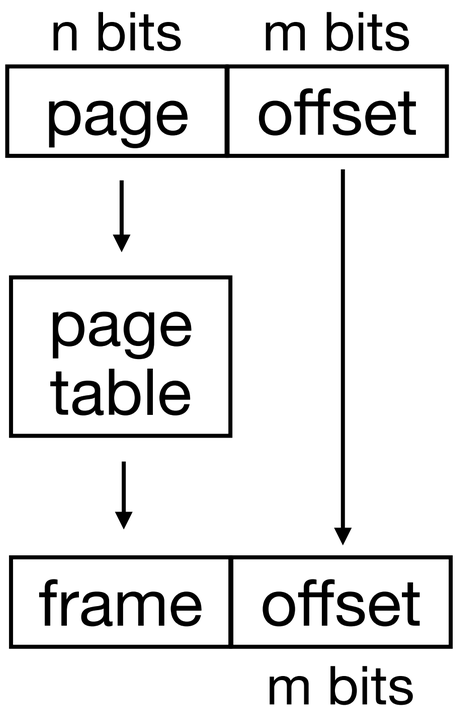
paging and segmentation



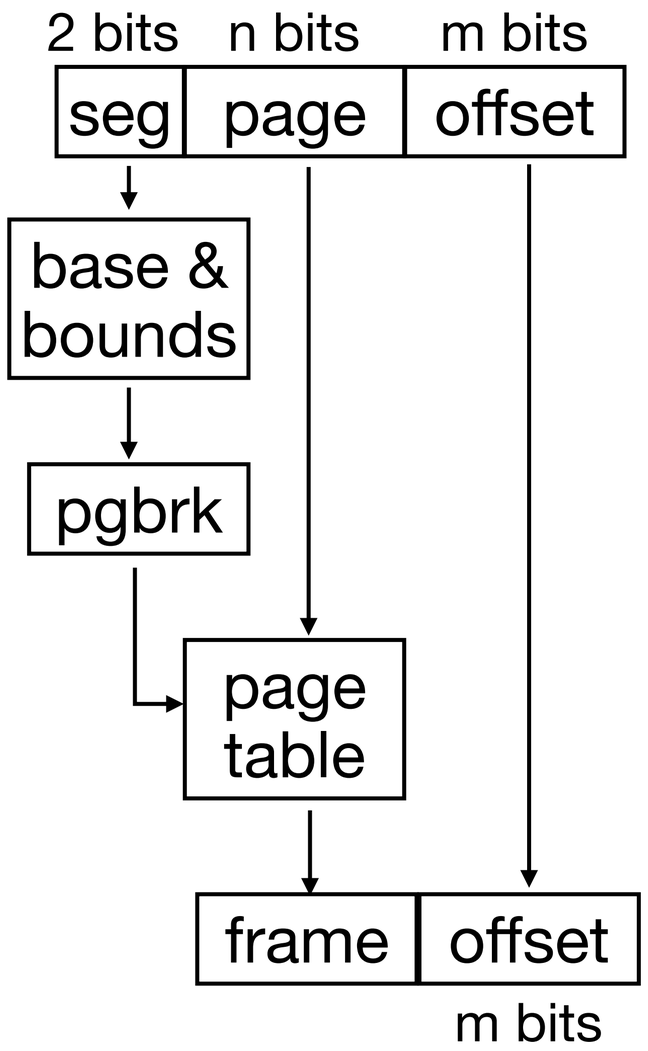
Paging + segmentation



segmentation



paging



paging + segmentation hybrid

Paging + segmentation

```
segment = (virtual_address & SEG_MASK) >> SEG_SHIFT
offset = virtual_address & OFFSET_MASK
if (offset > bounds[segment])
    RaiseException(PROTECTION_FAULT)
else
    phys_addr = base[segment] + offset
    register = AccessMemory(phys_addr)
```

segmentation

find base and bounds for the segment

check if offset is within the bounds

add offset to the base to get physical address

```
segment = (virtual_address & SEG_MASK) >> SEG_SHIFT
VPN = (virtual_address & OFFSET_MASK) >> VPN_SHIFT
PTEAddr = base[segment] + (VPN * sizeof(PTE))
if (PTEAddr > bounds[segment]) RaiseException(PROTECTION_FAULT)
PTE = AddressMemory(PTEAddr)
if (!PTE.Valid) RaiseException(SEGFAULT)
else if (!CanAccess(PTE.ProtectBits)) RaiseException(PROTECTION_FAULT)
else
    offset = virtual_address & OFFSET_MASK
    phys_addr = (PTE.PFN << PFN_SHIFT) | offset
    register = AccessMemory(phys_addr)
```

paging + segmentation

get base and bounds for the page table

get a page table entry (PTE), check if it is valid

get a physical address by combining frame number with offset

Page tables

- Problem: Page tables are too big and use too much memory
- Option #1: larger pages
- Option #2: **segmentation for page tables**
  - Pros: works with any page table size, less internal fragmentation
  - Cons: complexity - 3 (smaller) page tables per process
  - Pros: saves memory

Page tables

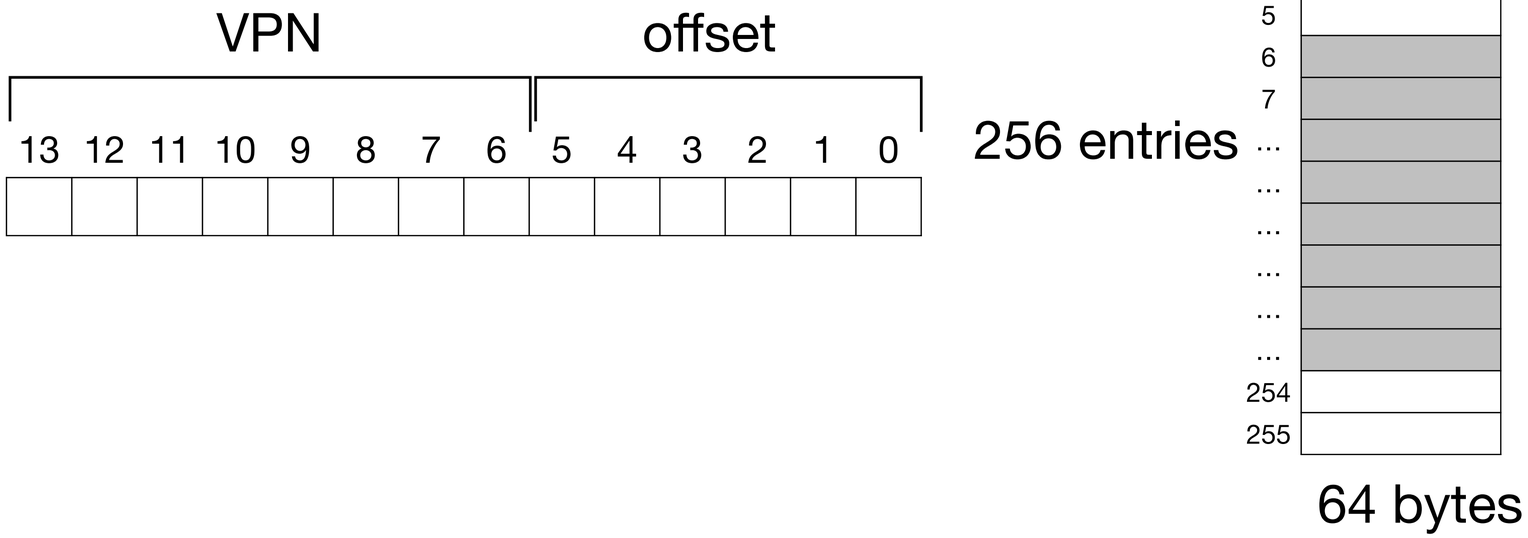
- Problem: Page tables are too big and use too much memory
- Option #1: larger pages
- Option #2: segmentation for page tables
- Option #3: **multi-level page tables** - turn page table into a tree-like structure

Page tables

16KB address space = 16,384 = 2^14 - need 14 bits

64-byte page = 2^6 bits for offset

256 page table entries = 2^8 bits
for virtual page number (VPN)

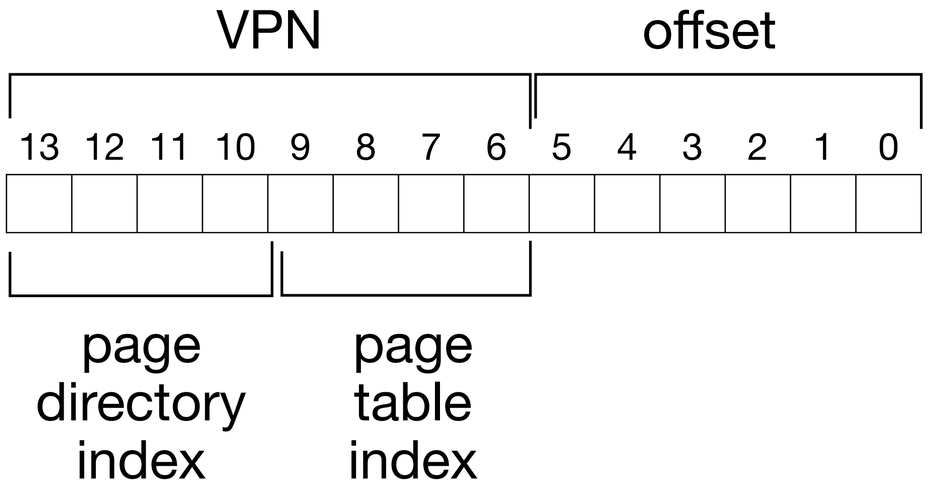


Multi-level page tables

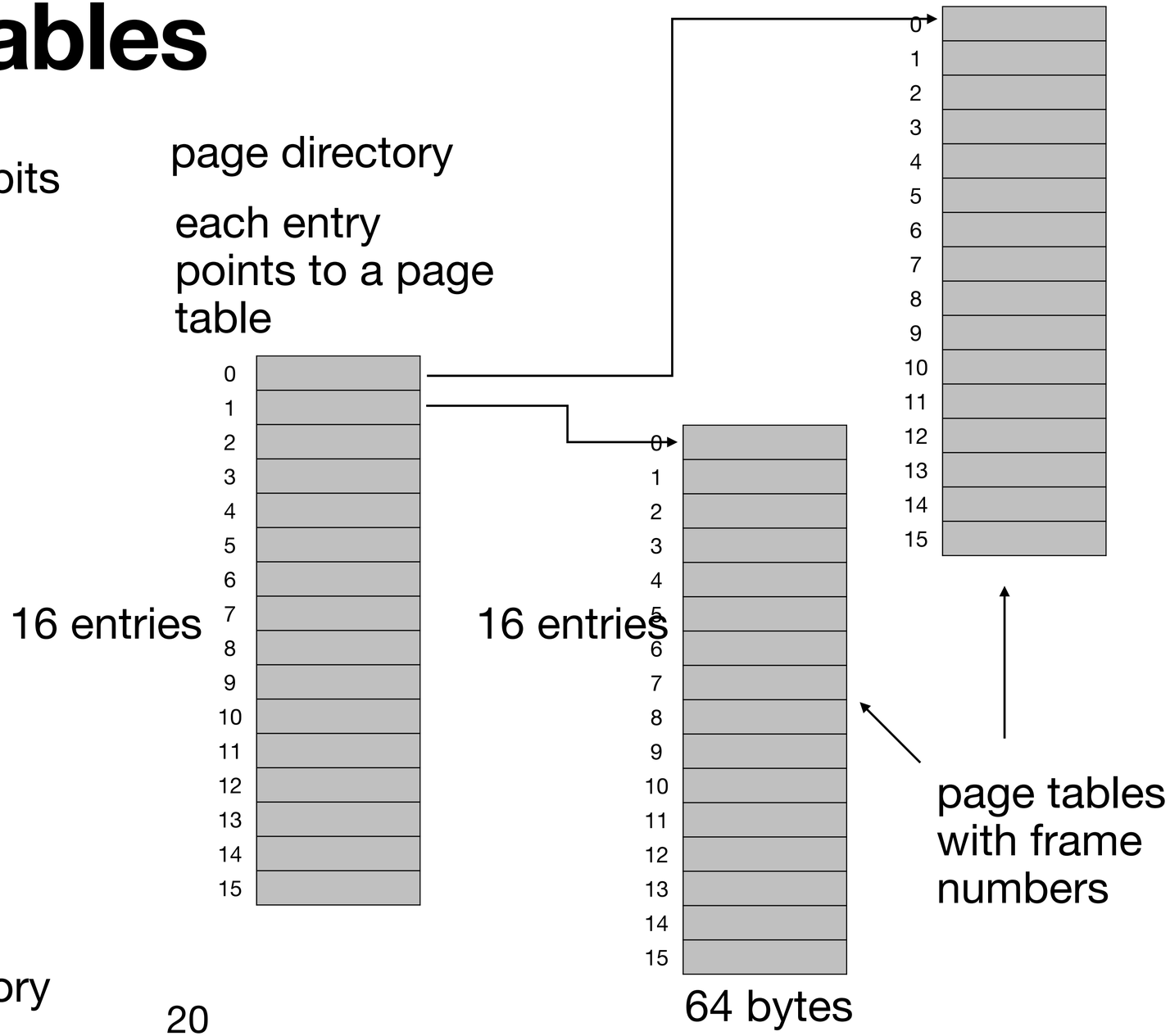
16KB address space = 2^14 - need 14 bits

64-byte page = 2^6 bits for offset

256 page table entries = 2^8 bits
for virtual page number (VPN)



use 4 bits for 16 entries in the page directory

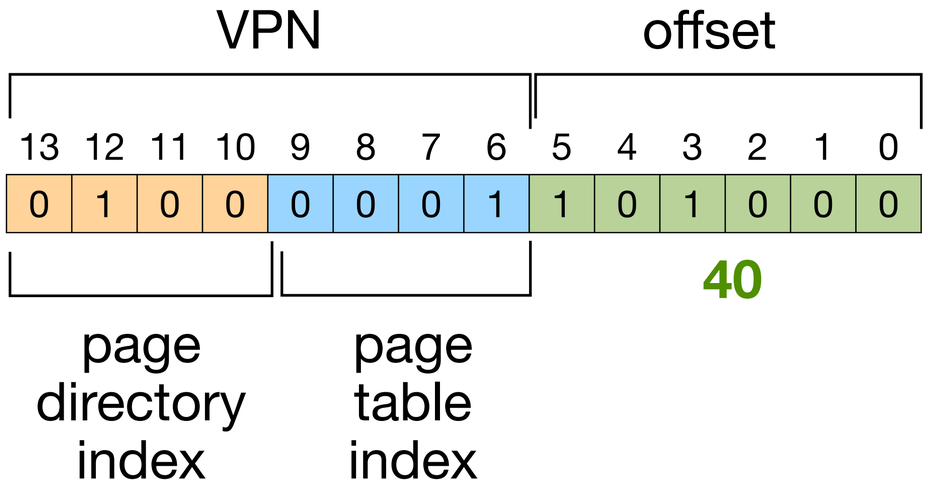


Multi-level page tables

16KB address space = 2^14 - need 14 bits

64-byte page = 2^6 bits for offset

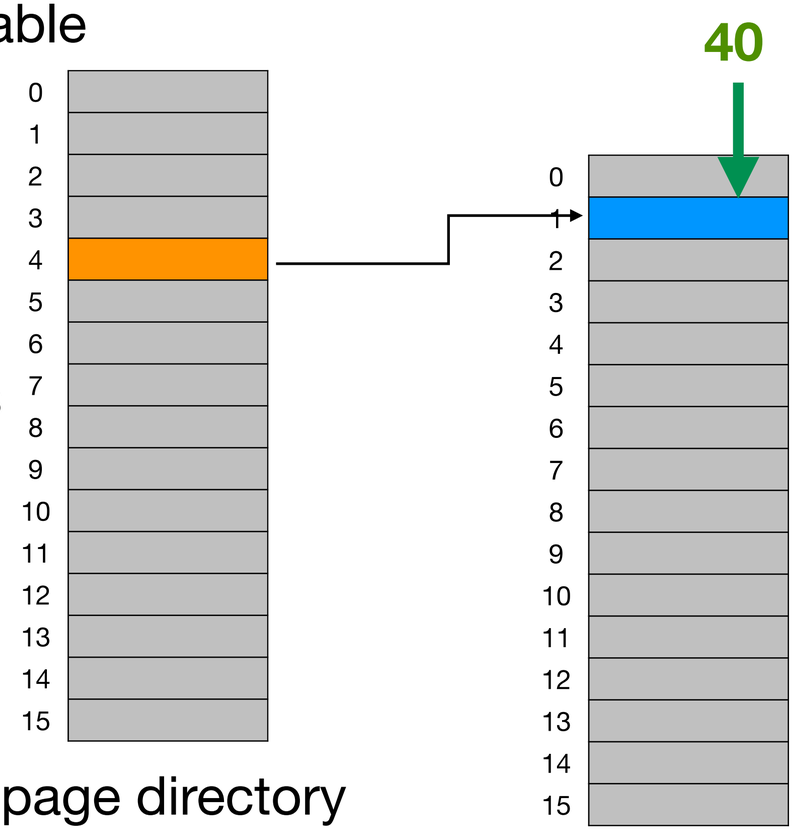
256 page table entries = 2^8 bits
for virtual page number (VPN)



16 entries

page directory

each entry
points to a page
table

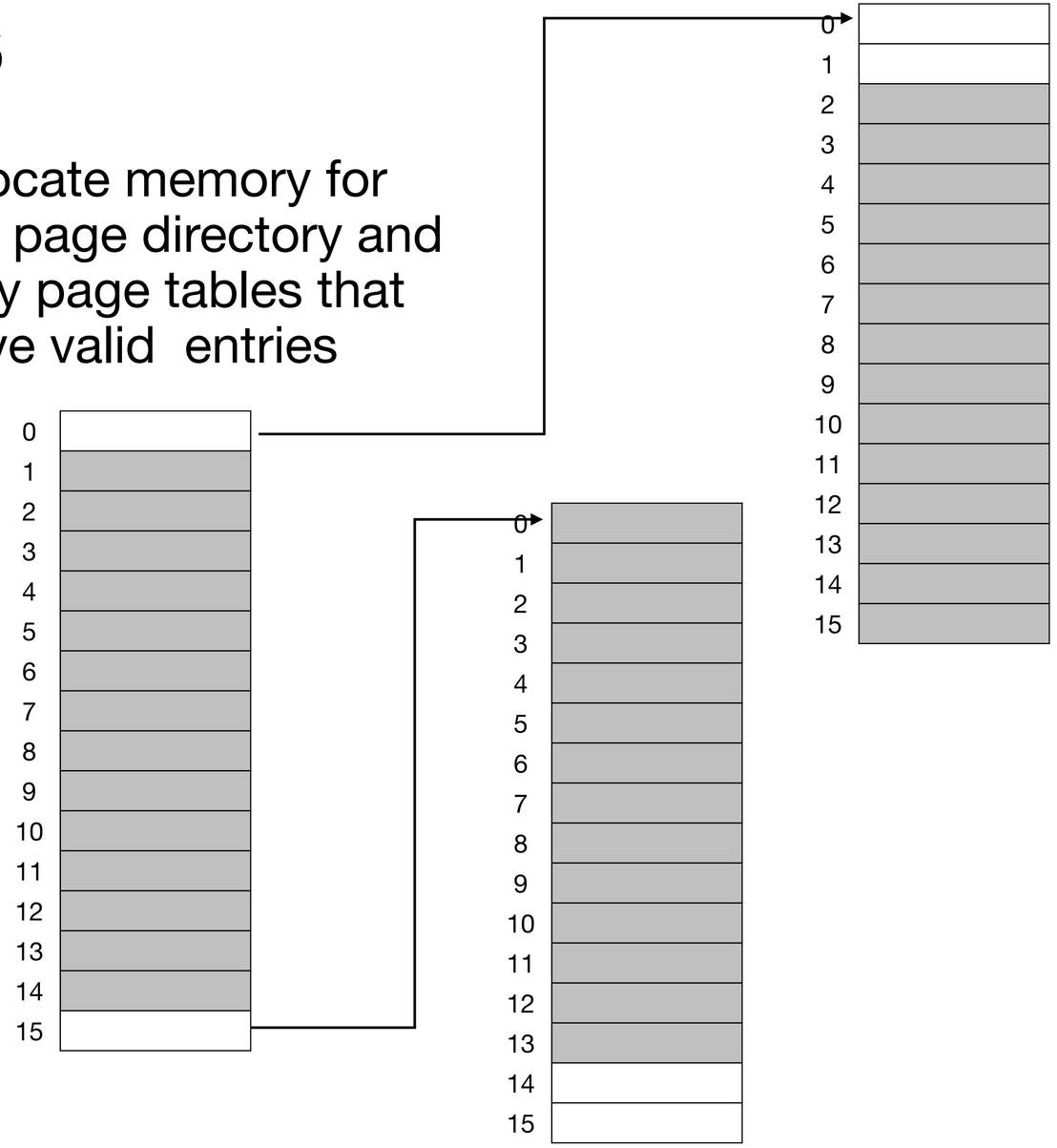


use 4 bits for 16 entries in the page directory

Multi-level page tables

- Make page tables **smaller**
- Allow allocating memory only for used pages
- Good for **sparse** address spaces
- Works with **TLB**
- Cons:
  - memory accesses are **slower**

Allocate memory for the page directory and only page tables that have valid entries



64 bytes

Multi-level page tables

```
VPN = (virtual_address & OFFSET_MASK) >> SHIFT
(success, TlbEntry) = TLB_Lookup(VPN)
if (success == True) //TLB Hit
    if (CanAccess(TlbEntry.ProtectBits) == True)
      offset = virtual_address + OFFSET_MASK
      physical_address = (TlbEntry.PFN << SHIFT) | Offset
      register = AccessMemory(physical_address)
    else RaiseException(PROTECTION_FAULT)
```

Get the virtual page number and check if it's in the TLB

If it is (TLB hit), then get get the frame, combine it with the offset and access the memory

Multi-level page tables

worst case still 2x memory operations

```
    // continued from the previous slide
VPN = (virtual_address & OFFSET_MASK) >> SHIFT
else //TLB Miss
PDIndex = (VPN & PD_MASK) >> PD_SHIFT
PDEAddr = PDBR + (PDIndex * sizeof(PDE))
PDE = AccessMemory(PDEAddr)
if (!PDE.Valid) RaiseException(SEGFAULT)
else
    PTIndex = (VPN & PT_MASK) >> PT_SHIFT
    PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
    PTE = AccessMemory(PTEAddr)
    if (PTE.Valid == False) RaiseException(SEGMENTAION_FAULT)
    else if (CanAccess(PTE.ProtectBits) == False)
      RaiseException(PROTECTION_FAULT)
    else
      TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
      RetryInstruction()
```

if the virtual page number is not in the TLB, we have a miss

first access the page directory

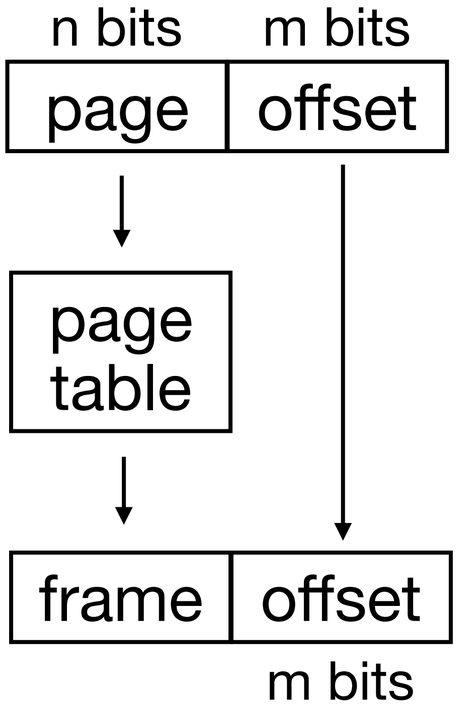
then get the corresponding page table

get the physical frame

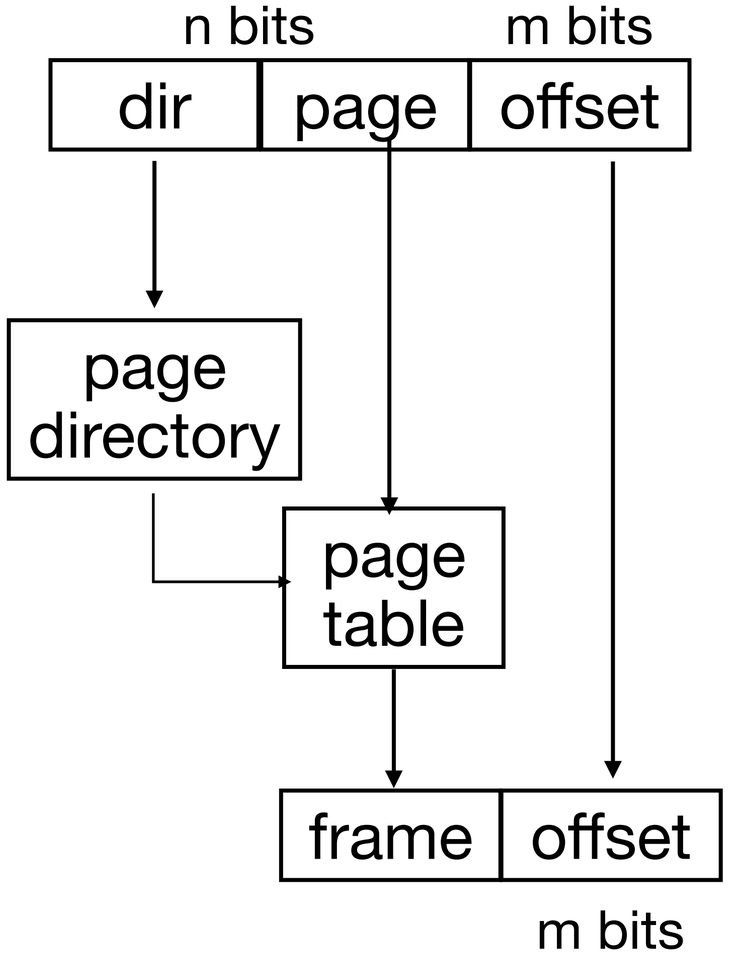
insert the frame into the TLB

and re-do the instruction, this time it should be a TLB hit

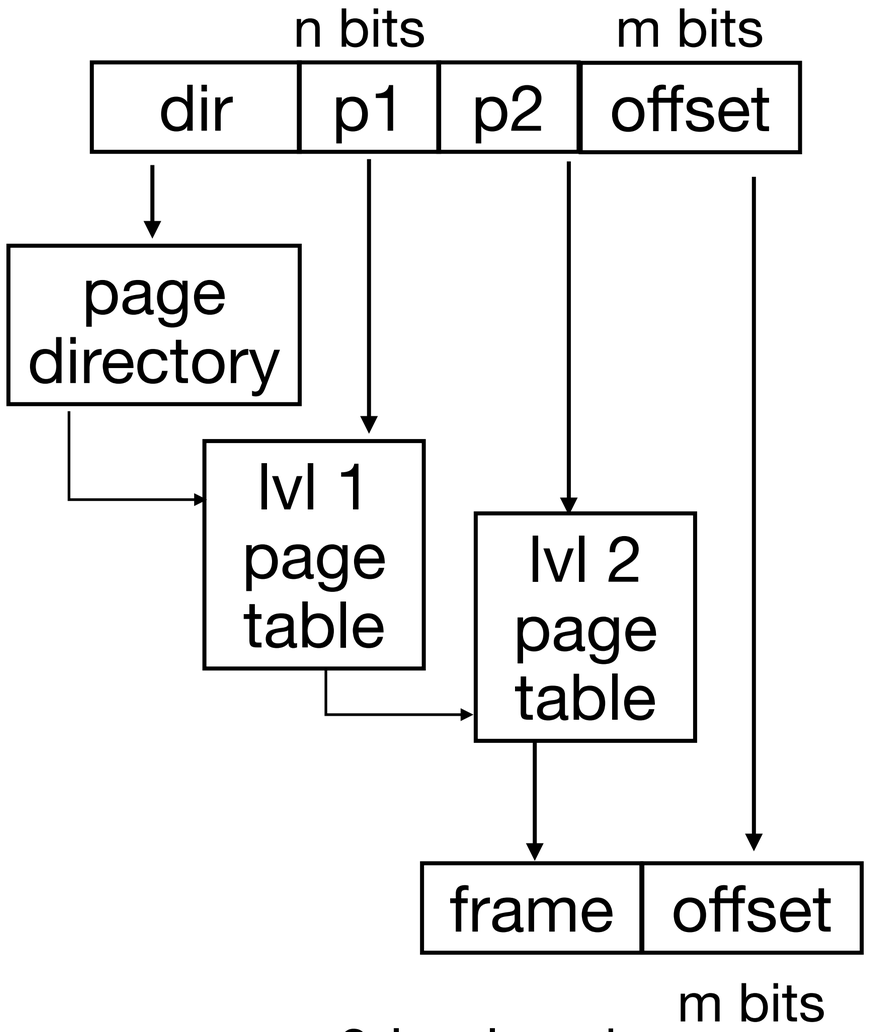
Multi-level page tables



paging



2-level paging



3-level paging

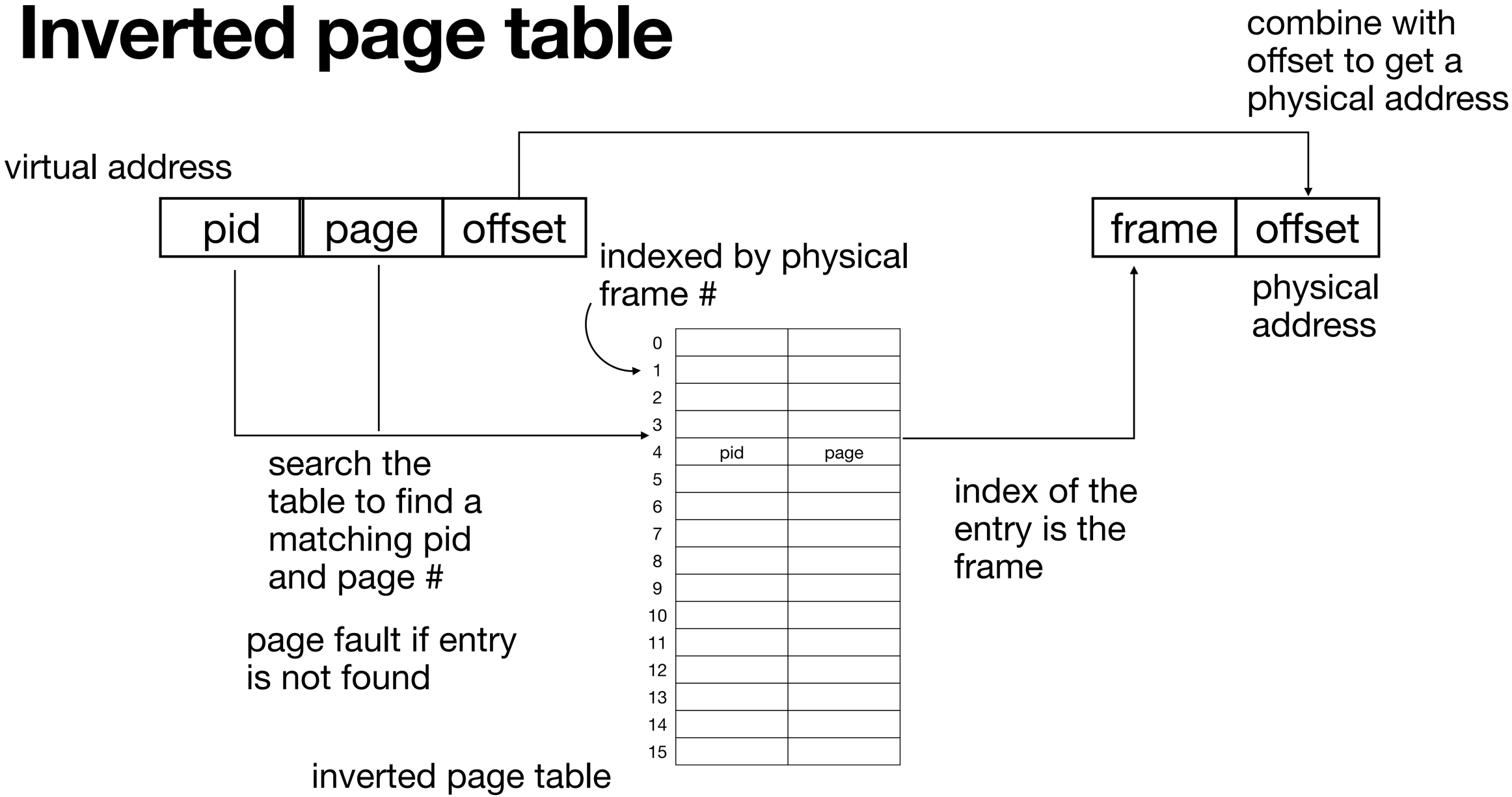
Multi-level page tables

- This saves **a lot** of memory
- 64-bit virtual address space with 4KB pages and 4GB of physical memory
- Single-level page table will use approx. 2^54 bytes or **~16 petabytes per process**
- 6-level page table can bring the page table size to **4KB per process**
- But will require **6x memory accesses** in the **worst case**

Page tables

- Problem: Page tables are too big and use too much memory
- Option #1: larger pages
- Option #2: segmentation for page tables
- Option #3: multi-level page tables - turn page table into a tree-like structure
  - Pros: saves a lot of memory
  - Cons: complexity - more memory operations
- Option #4: **inverted page tables** - index physical pages - one table for all processes

Inverted page table



Inverted page table

- **Index physical pages** instead of virtual ones
- Pros:
  - 1 page table for all processes
- Cons:
  - table **is large** - proportional to the size of RAM - search can take **a while**
    - hash table + TLB can improve performance
  - hard to do **shared memory** (multiple pids match to the same physical address)

Page tables

- Problem: Page tables are too big and use too much memory
- Option #1: larger pages
- Option #2: segmentation for page tables
- Option #3: multi-level page tables - turn page table into a tree-like structure
- Option #4: **inverted page tables** - index physical pages - one table for all processes
  - Pros: saves memory - only one large table for all processes
  - Cons: search is expensive, not easy to make work with shared pages

Paging in the wild

- Most architectures support both multi-level page tables and segmented pages
- But most OSs use **multi-level page tables**
  - e.g. Linux 64-bit systems implement 4-level paging
- Inverted page tables were more common in the 1990s
  - PowerPC, UltraSPARC, IA-64

Summary

- Problem: How to **reduce memory** required for storing **page tables**?
- Option #1: **larger pages** - reduce page table size, but lead to internal fragmentation
- Option #2: **segmentation for page tables** - no internal fragmentation, but issues with complexity
- Option #3: **multi-level page tables** - turn page table into a tree-like structure
  - less internal fragmentation, but high complexity and slow memory access
- Option #4: **inverted page tables** - index physical pages - one table for all processes
  - save memory, but issues with search and processes sharing memory

Reading

- Chapter 18 Paging: Smaller pages
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-smalltables.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-smalltables

- No optional reading this week

A2

- Due on **Feb 15 @ 11:59 PM EST**
- Start ASAP
- Few notes for development:
  - you **do not** need a remote server
    - just clone the gitlab repo, **develop locally,** and push to get feedback
    - **do not** try docker setup if you are short on time
  - get the autograder running **first**
  - push changes **frequently** - will help identify and fix problems **early**
    - tests are in the order of increasing complexity
    - there is **no limit** on the number of autograder reports
  - write a decent README to get an 8/10

A2 LOC test

- **do not** optimize before the code passes all tests
  - **premature optimization** will make your code **much harder to debug**
  - write the **straightforward code** that is easy to debug
  - you can get **80%** with no additional optimization

A2 LOC test

- Compact code test
  - cloc will be used to count number of lines in freelist.c and freelist.h
    - if you are not using the header, leave it empty
    - empty lines and comments are **not counted**, **do not** remove them from your code
  - **do not try to game the metric**, follow the style guidelines in A2 instructions
    - do not combine multiple lines with a semicolon (;)
    - put closing curly bracket (}) on a new line
    - put curly brackets ({}) around if..else
    - **ask if unsure** to avoid being disqualified
    - **if you feel that you are trying to cheat the metric, you probably are**
  - install and run cloc locally to measure LOC of your implementation

Midterm

- Feb 26 during class time
- In-person, closed book
- **60% multiple choice** - 20 MCQ
  - similar but **not identical** to in-class quizzes
- **40% short answers** - 5 short answer questions
  - similar but **not identical** to lab and lecture exercises
    - e.g. CPU scheduling, identifying/fixing errors in C code, addressing

Midterm

- How to prepare
  - **repetition** - attend classes, do quizzes after classes, do the labs
  - **recall** - practice quizzes and review lectures
    - **do not memorize** definitions, learn to apply them
      - e.g. if forked processes are independent from their parents, what does it mean in practice
    - **do not memorize** quiz answers, questions will never repeat
    - for each answer option, try to **explain in your own words** why it is correct/incorrect. If you don’t know, search or ask

Grades

- Be strategic about grades
  - do the **in-class quizzes** - **extra 5%** to compensate for mistakes on assignments (also helps with midterm prep)
  - do the **labs** - **easy 10%** of the final grade (also helps with midterm prep)
  - **50% avg** on assignments is needed for passing the course
    - assignments are designed to be **easily passable**
      - e.g. A1, you'd get >50% for RR without context switch + README
      - in A2, bump + first fit + README is >50%

Quiz 8

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999